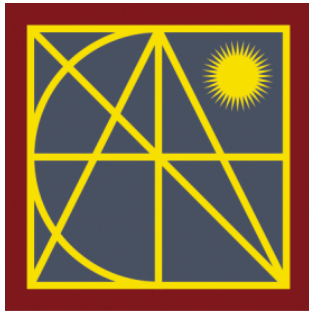




FaCENA - UNNE

Facultad de Ciencias Exactas
y Naturales y Agrimensura
**UNIVERSIDAD NACIONAL
DEL NORDESTE**

PARADIGMAS Y LENGUAJES TRABAJO PRACITO INTEGRADOR

SISTEMAS DE SEMÁFOROS INTELIGENTES Y ANÁLISIS
COMPARATIVOS DE PARADIGMAS

Integrantes:

- Gomez Matias Nicolas
- Arce Federico Hernán
- Maidana Adrián Ezequiel

Índice

Fundamento del diseño	3
Bitácora de bugs	3
Justificación de la fase 2	7
Análisis fase 3	7
Conclusión final	9
Bibliografía	10

Fundamento del diseño

Nuestro sistema es el resultado de la elaboración de los principios del paradigma funcional, se logro un resultado optimo evitando crear variables globales innecesarias, se tuvo detalle en no producir efectos secundarios sobre las estructuras creadas y evitando malas prácticas.

Nos basamos en los siguientes principios de la programación funcional:

Utilización de funciones puras. Durante la elaboración del código las funciones que mas predominaron en la solución son las de naturaleza pura, no afectan a otras variables, no modifican el entorno.

Composición. Se puede apreciar la forma en la que se puede utilizar una función ya existente dentro de otra para complementarse y llegar a la solución del problema, como por ejemplo las funciones duración-temporal que llama a contar-porcentaje-color para obtener la información separada de cada ocupación de los colores en el ciclo, y poder presentar toda la información en una lista.

Inmutabilidad. Dentro del sistema no existen funciones globales hardcodeadas, en su lugar las funciones dependen de argumentos que reciben como parámetros para devolver el resultado a la solución.

Bitácora de bugs

Depuración

- Se cambio la linea 31, para que sea adaptativa a los valores que se quieran poner de los colores, antes quedaba fijamente hard-codeado el valor 216 limitando a solo los valores dados por la consigna sobre la duracion de cada color

```
28 30
29 31 (defun timer (n)
30 32   (let ((duraciones '((rojo 90) (amarillo 6) (verde 120))))
31 33   - (timerAux (mod n 216) duraciones)
32 34   + (timerAux (mod n (reduce '+ (mapcar 'cadr duraciones))) duraciones)
33 35   )
34 36 )
```

Instalación librería Quicklisp

```
; undefined function: COMMON-LISP-USER::BUSCAR-FIN-CICLO
;
; compilation unit finished
; Undefined function:
;   BUSCAR-FIN-CICLO
; caught 1 STYLE-WARNING condition
;
; file: C:/Lisp/TPI-Funcional-2026-Grupo12/TPI-Funcional-2026-Grupo12/Lisp/core.lisp
; in: DEFUN CONTAR-COLOR
;   (COND ((>= SEGUNDO FIN) ACUM) ((EQUAL (SEMAFORO-TIMER SEGUNDO) COLOR))
;     (CONTAR-COLOR COLOR (+ SEGUNDO 1) FIN (+ ACUM 1)))
;     (T (CONTAR-COLOR COLOR (+ SEGUNDO 1) FIN ACUM)))
; --> OR LET IF
; ==>
;   (IF CONTAR-COLOR
;     (PROGN COLOR (+ SEGUNDO 1) FIN (+ ACUM 1))
;     (THE T (CONTAR-COLOR COLOR (+ SEGUNDO 1) FIN ACUM)))
;
; caught WARNING:
; undefined variable: COMMON-LISP-USER::CONTAR-COLOR
;
; compilation unit finished
; Undefined variable:
;   CONTAR-COLOR
; caught 1 WARNING condition
;
; * (registrarCambiosEstado 'rojo 'amarillo)
; Tiempo [2026-06-14 00:29:08]: la luz ha cambiado de ROJO a AMARILLO
; NIL
; *
```

Se instalo la libreria Quicklisp y se utilizó local-time, como CLISP 2.49 arrastra incompatibilidades con Windows 10/11 se utilizó para su funcionamiento SBCL (Steel Bank Common Lisp), logrando ejecutar correctamente la función por el cmd.

Solución a la incompatibilidad de la librería

```
[1]> (load "C:\\Lisp\\TPI-Funcional-2026-Grupo12\\TPI-Funcional-2026-Grupo12\\Lisp\\core")
;; Loading file C:\\Lisp\\TPI-Funcional-2026-Grupo12\\TPI-Funcional-2026-Grupo12\\Lisp\\core.lisp ...
*** - Error de Win32 267 (ERROR_DIRECTORY); The directory name is invalid.
Es posible continuar en los siguientes puntos:
SKIP      :R1      skip (LET # #)
RETRY     :R2      retry (LET # #)
STOP      :R3      stop loading file C:\\Lisp\\TPI-Funcional-2026-Grupo12\\TPI-Funcional-2026-Grupo12\\Lisp\\core.lisp
ABORT     :R4      Abort main loop
Break 1 [2]>
```

Modificación para poder ejecutar la biblioteca quicklisp para usar la función local time en la función registrar-cambios-estado.

```
1  [
2  {
3    "keys": ["alt+ctrl+h"],
4    "command": "repl_open",
5    "args": {
6      "type": "subprocess",
7      "encoding": "utf8",
8      //"cmd": ["C:\\clisp-2.49\\clisp"],
9      "cmd": ["C:\\Program Files\\Steel Bank Common Lisp\\sbcl.exe"],
10     "cwd": "C:\\Lisp",
11     "external_id": "lisp",
12     "syntax": "Packages/Lisp/Lisp.tmlanguage"
13   }
14 }
15 ]
```

Función ejecutándose correctamente

```
;
; compilation unit finished
; Undefined function:
;   BUSCAR-FIN-CICLO
; caught 1 STYLE-WARNING condition
T
* (registrarCambiosEstado 'rojo 'amarillo)
Tiempo [2026-06-15 22:40:52]: la luz ha cambiado de ROJO a AMARILLO
NIL
* |
```

Salida función transición

```
* (transicion 'en-rojo 'rojo-intermitente)
(EN-ROJO CAMBIAR-A ROJO-INTERMITENTE)
*
```

Se cambio esta línea “(list colorActual 'cambiar-a cambiarA))” por “(list colorActual (format nil "Cambiar-a-~a" cambiarA))) ya que generaba una lista con 3 elementos que no era la salida esperada por la consigna.

Incompatibilidad con el nombre de la función timer

```
This is SBCL 2.6.5, an implementation of ANSI Common Lisp.
More information about SBCL is available at <http://www.sbcl.org/>.

SBCL is free software, provided as is, with absolutely no warranty.
It is mostly in the public domain; some portions are provided under
BSD-style licenses. See the CREDITS and COPYING files in the
distribution for more information.
* (load "C:/Lisp/TPI-Funcional-2026-Grupo12/TPI-Funcional-2026-Grupo12/Lisp/core")
To load "local-time":
  Load 1 ASDF system:
    local-time
; loading "local-time"

While evaluating the form starting at line 47, column 0
  of #P"C:/Lisp/TPI-Funcional-2026-Grupo12/TPI-Funcional-2026-Grupo12/Lisp/core.lisp":

debugger invoked on a SYMBOL-PACKAGE-LOCKED-ERROR in thread
#<THREAD tid=1188 "main thread" RUNNING (1104030083)>:
  Lock on package SB-EXT violated when proclaiming TIMER as a function while in
  package COMMON-LISP-USER.
See also:
  The SBCL Manual, Node "Package Locks"

Type HELP for debugger help, or (SB-EXT:EXIT) to exit from SBCL.

restarts (invokable by number or by possibly-abbreviated name):
  0: [CONTINUE] Ignore the package lock.
  1: [IGNORE-ALL] Ignore all package locks in the context of this operation.
  2: [UNLOCK-PACKAGE] Unlock the package.
  3: [RETRY] Retry EVAL of current toplevel form.
  4: Ignore error and continue loading file "C:\\Lisp\\TPI-Funcional-2026-Grupo12\\TPI-Funcional-2026-Grupo12\\Lisp\\core.lisp".
  5: [ABORT] Abort loading file "C:\\Lisp\\TPI-Funcional-2026-Grupo12\\TPI-Funcional-2026-Grupo12\\Lisp\\core.lisp".
  6: Exit debugger, returning to top level.

(PACKAGE-LOCK-VIOLATION #<PACKAGE "SB-EXT"> :SYMBOL TIMER :FORMAT-CONTROL "proclaiming ~S as a function" :FORMAT-ARGUMENTS (TIMER))
0]
```

El problema es un choque de nombres con la estructura de SBCL ya que existe **SB-EXT:TIMER** y ese símbolo vive en un paquete bloqueado. SBCL detecta que estarías "llamando" como función un símbolo que pertenece a un paquete locked, y no permite compilar.

Solución se agregó `#+sbcl (sb-ext:unlock-package :sb-ext)` al comienzo del archivo, esto desbloquea el paquete SB-EXT y permite compilar.

Justificación de la fase 2

Como grupo de hemos selecciona implementar local-time de la librería Quicklisp. El motivo de la selección fue que como estamos aprendiendo a usar Quicklisp y a gestionar dependencias externas en Common Lisp, preferimos elegir una librería con un propósito claro. Esto nos permitió concentrarnos en entender cómo funciona el gestor de paquetes y cómo importar librerías al núcleo de nuestro sistema. Además, nos pareció útil lograr incorporar datos reales como es la hora y el día específicos extraídos de información del exterior para su utilización en el sistema, nos parece una práctica muy común en cualquier proyecto que tengamos que afrontar y un conocimiento básico necesario para cualquier momento laborar que nos pueda surgir en el futuro.

Análisis fase 3

Estudio comparativo:

Presentación breve del lenguaje y en qué industrias o áreas se utiliza:

Erlang es un lenguaje de programación funcional, concurrente y de propósito general, diseñado originalmente por Ericsson en la década de 1980 para desarrollar sistemas de telecomunicaciones que debían funcionar de manera continua, incluso ante errores o caídas de componentes.

Se utiliza principalmente en:

Telecomunicaciones: Centralitas telefónicas y conmutadores de red como 3G/4G/5G.

Mensajería: Plataformas como WhatsApp y Facebook Messenger.

Banca y Fintech: Sistemas de pagos en tiempo real y plataformas de trading.

Comercio Electrónico: Motores de ventas y gestión de inventario distribuido.

1. Erlang utiliza una sintaxis derivada de Prolog. Muestren cómo se delimitan las funciones y las expresiones (puntos, comas y puntos y comas) y cómo afectó esto a la lectura del código comparado con los paréntesis de Lisp.

El punto termina la función. El punto finaliza una declaración. Se utiliza para cerrar una función o módulo.

El punto y coma separa las cláusulas u opciones. Se utiliza para separar diferentes "cláusulas" o caminos de una misma función.

La coma separa las expresiones. Se utiliza para separar las líneas de código (expresiones) que se ejecutan secuencialmente dentro de una cláusula.

La estructura es separada y más sencilla de leer; el uso de comas, puntos y punto y coma ayuda a separar visualmente las alternativas, lo que evita la molestia de contar o emparejar paréntesis.

2. En Erlang las variables son de *asignación única*. ¿Cómo simulan un ciclo continuo de cambios en el semáforo si no pueden reasignar valores a las variables?

Se simula un ciclo usando recursión y/o procesos que se ejecutan en bucle. Cada iteración crea nuevas variables, representando el nuevo estado del semáforo.

Ejemplo elaboración propia para nuestro código:

```
semaforo_timer(N) ->
```

```
    Duraciones = [
```

```
        {rojo, 90},
```

```
        {rojo_intermitente, 3},
```

```
        {amarillo, 6},
```

```
        {amarillo_intermitente, 3},
```

```
        {verde, 120},
```

```
        {verdeintermitente, 3}
```

```
    ],
```

```
    Tiempos = lists:map(fun({_, Tiempo}) -> Tiempo end, Duraciones),
```

```
    TotalCiclo = lists:foldl(fun(X, Sum) -> X + Sum end, 0, Tiempos),
```

```
    semaforo_timer_aux(N rem TotalCiclo, Duraciones).
```

```
semaforo_timer_aux(N, [{Color, Duracion} | Resto]) ->
```

```
    if
```

```
        N < Duracion ->
```

```
            Color;
```

```
        true ->
```

```
            semaforo_timer_aux(N - Duracion, Resto)
```

```
    end.
```

Conclusión final

La elaboración de las diferentes funciones nos permitió tener mas experiencia y practica en lo que conlleva la programación funcional, sus ventajas y desventajas en su uso, también nos brindo una oportunidad de poder tener más experiencia en trabajos en equipo, las herramientas ideales para la organización como fue GitHub y conocer a nuestros compañeros, además de aprender de ellos al compartir nuestros conocimientos y ayudarnos durante los errores.

Como experiencia extra destacamos el aprendizaje al comparar y desarrollar las funciones en el lenguaje que nos tocó “Erlang”, que nos permitió conocer de cerca una nueva tecnología que desconocíamos y que podría ser útil en el futuro, la sintaxis nos pareció interesante y cómoda, pero desafiante al estar acostumbrados a otras sintaxis de otras herramientas como LISP.

Para concluir la experiencia al elaborar este proyecto nos pareció muy interesante y enriquecedora, también aprendimos un poco más sobre la organización en equipos de trabajo, pudimos aplicar muchos conceptos aprendidos durante el cursado de la materia de paradigmas y lenguajes.

Bibliografía

<https://www.erlang.org/faq/introduction.html>

<https://www.erlang.org/faq/academic.html>

https://www-quicklisp-org.translate.goog/beta/? x tr sl=en& x tr tl=es& x tr hl=es& x tr_pto=tc

https://www.youtube.com/watch?v=zYDYYLbMj5Q&list=PLmoxkf3gS-juT5cDrzP_ARdkqgSFU5X_3

<https://lisp-docs.github.io/cl-language-reference/chap-21/cb-b-stream-concepts>